SIMATS ENGINEERING

ASSESSMENT TOOL 2 - MEDIUM (CASE STUDY QUESTIONS)

COURSE CODE: CSA09 COURSE NAME: Programming in Java COURSE OUTCOME COVERED

CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks: 50

ANNEXURE A

CASE STUDY QUESTIONS

Code of Conduct:

I K.GNANESWAR KUMAR - (192572105)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rubrics for Evaluation:

Criteria	Understanding of the Case	Application of Concepts	Analysis	Solution Design	Clarity	Total
Weightage	25%	25%	20%	20%	10%	
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

ANNEXURE

CASE STUDY QUESTIONS

1. Library Management System
Design a Java class to represent a library book. **Data Members:**

- Book ID
- Book Title
- Author Name
- Number of Copies Available

Methods:

1. Read book details.
2. Issue a book (decrease available copies by 1). 3. Return a book (increase available copies by 1). 4. Display updated book details.

Assume: Initial copies = 20.

2. Employee Salary Management
Design a Java class to represent an employee.
Data Members:

- Employee ID
- Employee Name
- Department
- Basic Salary

Methods:

1. Read employee details.
2. Calculate HRA (20% of Basic Salary).
3. Calculate DA (10% of Basic Salary).
4. Display gross salary.

Assume: Basic Salary = ₹25,000.

3. Student Fee Management System
Design a Java class to represent a student.
Data Members:

- Student ID
- Student Name
- Course
- Fee Balance

Methods:

1. Read student details.
2. Pay fee amount.
3. Check remaining fee balance.
4. Display student details.

Assume: Total Course Fee = ₹50,000.

4. Electricity Bill Management
Design a Java class to represent a consumer.
Data Members:
- Consumer Number
- Consumer Name
- Type of Connection (Domestic/Commercial) •
  Units Consumed

Methods:
1. Read consumer details.
2. Calculate electricity bill.
3. Display bill amount.
4. Validate connection type.

Assume:
- Domestic: ₹5 per unit
- Commercial: ₹8 per unit

5. Mobile Recharge System
Design a Java class to represent a mobile subscriber.
Data Members:
- Mobile Number
- Subscriber Name
- Service Provider
- Account Balance

Methods:
1. Read subscriber details.
2. Recharge account balance.
3. Deduct balance for a call/data usage.
4. Display updated balance.

Assume: Initial balance = ₹500.

ANSWERS:

1. Library Management System

```
import java.util.Scanner;

class LibraryBook {
   int bookId;
   String bookTitle;
   String authorName;
   int copiesAvailable = 20;

   void readDetails() {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter Book ID: ");
      bookId = sc.nextInt();
      sc.nextLine();

      System.out.print("Enter Book Title: ");
      bookTitle = sc.nextLine();

      System.out.print("Enter Author Name: ");
      authorName = sc.nextLine();
   }

   void issueBook() {
      if (copiesAvailable > 0) {
         copiesAvailable--;
         System.out.println("Book Issued Successfully.");
      } else {
         System.out.println("No copies available.");
      }
   }

   void returnBook() {
      copiesAvailable++;
      System.out.println("Book Returned Successfully.");
   }

   void displayDetails() {
      System.out.println("\nBook ID: " + bookId);
      System.out.println("Book Title: " + bookTitle);
      System.out.println("Author Name: " + authorName);
      System.out.println("Available Copies: " + copiesAvailable);
   }
```

```
  public static void main(String[] args) {
     LibraryBook b = new LibraryBook();
     b.readDetails();
     b.issueBook();
     b.returnBook();
     b.displayDetails();
  }
}
```

2. Employee Salary Management

```
import java.util.Scanner;

class Employee {
  int empId;
  String empName;
  String department;
  double basicSalary = 25000;

  void readDetails() {
     Scanner sc = new Scanner(System.in);

     System.out.print("Enter Employee ID: ");
     empId = sc.nextInt();
     sc.nextLine();

     System.out.print("Enter Employee Name: ");
     empName = sc.nextLine();

     System.out.print("Enter Department: ");
     department = sc.nextLine();
  }

  double calculateHRA() {
     return basicSalary * 0.20;
  }

  double calculateDA() {
     return basicSalary * 0.10;
  }

  void displayGrossSalary() {
     double grossSalary = basicSalary + calculateHRA() + calculateDA();

     System.out.println("\nEmployee ID: " + empId);
     System.out.println("Employee Name: " + empName);
     System.out.println("Department: " + department);
```

```
        System.out.println("Gross Salary: ₹" + grossSalary);
    }

    public static void main(String[] args) {
        Employee e = new Employee();
        e.readDetails();
        e.displayGrossSalary();
    }
}
```

3. Student Fee Management System

```
import java.util.Scanner;

class Student {
    int studentId;
    String studentName;
    String course;
    double feeBalance = 50000;

    void readDetails() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Student ID: ");
        studentId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Student Name: ");
        studentName = sc.nextLine();

        System.out.print("Enter Course: ");
        course = sc.nextLine();
    }

    void payFee(double amount) {
        feeBalance -= amount;
        System.out.println("Fee Paid Successfully.");
    }

    void checkBalance() {
        System.out.println("Remaining Fee Balance: ₹" + feeBalance);
    }

    void displayDetails() {
        System.out.println("\nStudent ID: " + studentId);
        System.out.println("Student Name: " + studentName);
        System.out.println("Course: " + course);
        System.out.println("Fee Balance: ₹" + feeBalance);
```

```
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.readDetails();
        s.payFee(10000);
        s.checkBalance();
        s.displayDetails();
    }
}
```

4. Electricity Bill Management

```
import java.util.Scanner;

class Consumer {
    int consumerNo;
    String consumerName;
    String connectionType;
    int unitsConsumed;

    void readDetails() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Consumer Number: ");
        consumerNo = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Consumer Name: ");
        consumerName = sc.nextLine();

        System.out.print("Enter Connection Type (Domestic/Commercial): ");
        connectionType = sc.nextLine();

        System.out.print("Enter Units Consumed: ");
        unitsConsumed = sc.nextInt();
    }

    boolean validateConnectionType() {
        return connectionType.equalsIgnoreCase("Domestic") ||
               connectionType.equalsIgnoreCase("Commercial");
    }

    double calculateBill() {
        if (connectionType.equalsIgnoreCase("Domestic"))
            return unitsConsumed * 5;
        else if (connectionType.equalsIgnoreCase("Commercial"))
            return unitsConsumed * 8;
```

```
        else
            return 0;
    }

    void displayBill() {
        if (validateConnectionType()) {
            System.out.println("Bill Amount: ₹" + calculateBill());
        } else {
            System.out.println("Invalid Connection Type.");
        }
    }

    public static void main(String[] args) {
        Consumer c = new Consumer();
        c.readDetails();
        c.displayBill();
    }
}
```

5. Mobile Recharge System

```
import java.util.Scanner;

class MobileSubscriber {
    String mobileNumber;
    String subscriberName;
    String serviceProvider;
    double accountBalance = 500;

    void readDetails() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Mobile Number: ");
        mobileNumber = sc.nextLine();

        System.out.print("Enter Subscriber Name: ");
        subscriberName = sc.nextLine();

        System.out.print("Enter Service Provider: ");
        serviceProvider = sc.nextLine();
    }

    void recharge(double amount) {
        accountBalance += amount;
        System.out.println("Recharge Successful.");
    }

    void deductBalance(double amount) {
```

```
        if (accountBalance >= amount) {
            accountBalance -= amount;
            System.out.println("Balance Deducted.");
        } else {
            System.out.println("Insufficient Balance.");
        }
    }

    void displayBalance() {
        System.out.println("Updated Balance: ₹" + accountBalance);
    }

    public static void main(String[] args) {
        MobileSubscriber m = new MobileSubscriber();
        m.readDetails();
        m.recharge(200);
        m.deductBalance(100);
        m.displayBalance();
    }
}
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand